Git for Solo Devs: A Practical, Opinionated Workflow

> The Git workflow for working alone — not the team-collaboration workflow you read about in every other tutorial. Includes history-rewriting, safe recovery, branching, and the few git config flags that pay for themselves immediately.
>
> **NOTE. LAST VALIDATED: 2026-05.** Git 2.43+ (modern `switch`/`restore` semantics, `push.autoSetupRemote`), `gh` CLI 2.40+. Bump and re-validate annually per CLAUDE.md standard #5.

What you'll have at the end

- A clear workflow for day-to-day solo development: branch, commit, merge, delete.
- The history-rewriting toolkit and when to use each tool (`commit --amend`, `rebase -i`, `reset`, `revert`).
- Confidence to use `--force-with-lease` deliberately on your own branches without burning anything.
- A mental model of `git reflog` as the safety net that means **almost nothing in git is truly lost**.
- Concrete recovery procedures for the most common "oh no" moments: committed to wrong branch, force-pushed bad code, deleted a branch you needed, lost work in a botched rebase.
- A `~/.gitconfig` with the few aliases and settings that compound across years of use.

Prerequisites

- Git 2.40+ (any modern install).
- The `gh` CLI (`gh` from cli.github.com — strongly recommended; some commands below use it).
- VS Code with the **GitLens** extension if you want a visual layer (optional but useful).
- Comfort with the command line.

Table of contents

---

1. The principle: solo git is not team git

Most Git tutorials are written for teams. They optimize for:

- **Many people pushing simultaneously** — hence "never rewrite shared history."
- **Code review as a gate** — hence "PRs are required, force-merge is forbidden."
- **Long-lived branches** — hence GitFlow, develop/release/hotfix branches.
- **Audit trails for compliance** — hence merge commits are sacred.

When you're solo, **almost none of that applies**. The constraints flip:

- **You're the only one pushing.** Rewriting your own history is fine; you can't "break" anyone else.
- **No code review gate.** Or rather: you are the reviewer, and your tools (the diff, the commit log, the linter) are what stand between you and broken main.
- **Long-lived branches are pure overhead.** They diverge from main, accumulate conflicts, and slow you down.
- **History is for your future self** to navigate when something breaks at 11pm six months from now.

So the workflow flips too. Solo Git is:

- **Tiny branches**, lived for a feature, then deleted.
- **Clean linear history** — rebase, not merge commits, for incoming work.
- **Aggressive history rewriting** before push. Push is the publication point.
- **`main` is always deployable.** Every merge goes to production.

The remaining "team" pieces — pull requests, code review, merge conflicts — still have value, but for **different reasons** than in a team:

- PR descriptions become **changelogs for future-you**.
- Self-review with `gh pr diff` catches things you missed.
- Conflicts mostly come up when you've been working across machines.

> **IMPORTANT. `MAIN` IS SACRED. EVERYTHING ELSE IS YOURS TO REWRITE.** That's the whole philosophy compressed to one line. Once code is on *`main`* and pushed, treat it as immutable. While it's on a feature branch, rewrite freely.

---

2. The minimum workflow

For 90% of solo work, this is all you need:

```
# start
git checkout main
git pull --rebase
git checkout -b feature/leaderboard

# work
# ... edit files ...
git add -A
git commit -m "add leaderboard skeleton"
# ... edit files ...
git add -A
git commit -m "wire leaderboard to scores table"

# clean up the history before pushing (optional but encouraged)
git rebase -i main

# push and PR
git push -u origin feature/leaderboard
gh pr create --fill

# self-review the diff (in browser or terminal)
gh pr diff

# merge
gh pr merge --auto --squash --delete-branch

# back to main
git checkout main
git pull
```

That's it. Branches are short (hours to a day), commits are small, history is clean before it lands.

Why `--squash` on merge: because the individual commits in a feature branch are scaffolding for you while building. Once merged, future-you only cares about the feature as a unit. Squash gives you one clean commit per feature on `main`.

Why `--delete-branch`: solo branches are throwaway. Don't accumulate them.

Why `--rebase` on pull: keeps history linear. Avoids the "merge commit just to pull main into my branch" noise.

> **TIP.** Use *`git switch`* instead of *`git checkout`* for branches, and *`git restore`* for files. They were split out in Git 2.23 (2019) specifically because *`git checkout`* was overloaded to do too many things and people kept conflating "switch branches" with "discard file changes." The new commands are unambiguous.
>
> *`bash git switch main # switch to existing branch git switch -c feature/new # create + switch (equivalent of checkout -b) git restore path/to/file # discard uncommitted changes to that file git restore --staged file # unstage (move from index back to working tree)`*

---

3. Branching: when and when not

Always branch when:

- You're writing more than a one-line fix.

- The work might take more than one work session (so you can leave it half-done without polluting main).
- You'd want CI to run before merging.
- You want a PR description as a changelog entry.

Skip the branch when:

- It's a typo fix or doc-only change that you'll commit + push immediately.
- The "branch" would live for 30 seconds.

For everything in between, default to branching. The cost is tiny (`git switch -c`) and you gain the ability to bail out cleanly if the approach turns out wrong.

Branch naming

Match `prefix/short-description`. Prefixes signal intent:

Prefix	For
`feature/`	New functionality
`fix/`	Bug fixes
`refactor/`	Restructuring without behavior change
`docs/`	Documentation only
`chore/`	Tooling, dependencies, CI
`experiment/`	"I might throw this away" — useful flag for yourself

Solo, the prefix is mostly for your own context-switching. When you look at `git branch` and see 4 branches, you want to remember what each one is at a glance.

Don't:

- Don't put issue numbers in branch names unless you have lots of issues. Adds noise.
- Don't put dates in branch names. You have `git log`.
- Don't use spaces, special characters, or capitals. `feature/add-leaderboard`, not `feature/Add Leaderboard`.

Branch lifetime

Solo branches should live **hours to a day**. If a branch is alive for a week, you have three problems:

1. It's diverged from `main` so much that rebase will be painful.
2. You've forgotten the context of the early commits.
3. You're missing the dopamine of shipping.

If you find a branch growing beyond a day, ask: can I ship what I have now (behind a feature flag, or as an obviously-WIP commit), and continue on top of `main`?

> **CAUTION. LONG-LIVED BRANCHES ARE A SOLO-DEV ANTI-PATTERN.** They're often a sign of "I'm not sure where I'm going" — which is fine, but solve that by shipping the smallest thing that works and iterating on *`main`*, not by accumulating uncommitted-pseudo-progress on a stale branch.

---

4. Writing commit messages (yes, still matters)

You're solo. So who reads your commit messages? **Future-you, six months from now, at 11pm, trying to figure out what changed.**

That alone justifies decent messages. Rules of thumb:

1. **Imperative mood, ~50 chars first line.** "Add leaderboard command," not "Added the leaderboard command" or "Adds leaderboard."
2. **Capitalize the first letter. No trailing period.** Convention.
3. **Body when needed, explaining why.** The diff shows what changed; the body explains why.
4. **One concept per commit.** If you'd say "and" describing it, it's two commits.
5. **No "fix typo" 30-commit chains.** Use `git commit --amend` or interactive rebase to fold them into the relevant commit before pushing.

Examples

× **Bad:**

```
fixed stuff
WIP
asdf
update
broken, will fix later
```

 Good:

```
Add /leaderboard slash command

Reads top 10 users from `scores` table, sorted by total_score
desc. Falls back to "no scores yet" if the table is empty.
```

 Also fine for trivial work:

```
Bump black to 24.10.0
```

Conventional commits

Some teams use Conventional Commits — `feat:`, `fix:`, `chore:` prefixes that drive automated changelog generation. **For solo work, skip this** unless you're using a tool that genuinely needs it (semantic-release, etc.). The prefixes add noise without value when you're the only one writing them.

---

5. The history-rewriting toolkit

Four tools. Use the right one for the job.

`git commit --amend`

Adds the staged changes to the **last commit**. Also lets you rewrite the last commit's message.

```
# Edit a file you forgot to include in the last commit
git add forgotten-file.py
git commit --amend --no-edit          # add changes, keep message
git commit --amend                     # add changes, edit message in editor
```

When to use: the most recent commit needs adjusting and hasn't been pushed yet.

> **WARNING. `--AMEND` REWRITES THE LAST COMMIT'S SHA.** If you already pushed, you'd need `--force-with-lease` to push again (see §6). Best practice: don't amend pushed commits unless you're certain you're the only one consuming the branch.

`git rebase -i <base>` (interactive rebase)

The Swiss army knife. Replays commits one at a time, letting you reorder, squash, edit, or drop each one.

```
git rebase -i main           # rewrite all commits since main
git rebase -i HEAD~5         # rewrite the last 5 commits
```

Git opens your editor with a list:

```
pick abc123 Add leaderboard skeleton
pick def456 Fix typo
pick ghi789 Wire leaderboard to scores
pick jkl012 Fix another typo
```

Change `pick` to:

- `reword` (`r`) — keep the commit, edit the message.
- `edit` (`e`) — stop on this commit, let you amend.
- `squash` (`s`) — combine with the previous commit, merge messages.
- `fixup` (`f`) — like squash but discard this commit's message.
- `drop` (`d`) — remove the commit entirely.

You can also reorder lines to reorder commits.

A typical pre-push cleanup turns this:

```
pick abc123 Add leaderboard skeleton
pick def456 Fix typo
pick ghi789 Wire leaderboard to scores
pick jkl012 Fix another typo
pick mno345 Add tests
```

Into this:

```
pick abc123 Add leaderboard skeleton
fixup def456 Fix typo
pick ghi789 Wire leaderboard to scores
fixup jkl012 Fix another typo
pick mno345 Add tests
```

Resulting in three clean commits instead of five noisy ones.

`git reset`

Moves the branch pointer (and optionally the index and working tree) to a different commit.

```
git reset --soft <commit>       # move branch; keep changes staged
git reset --mixed <commit>      # move branch; keep changes unstaged (default)
git reset --hard <commit>       # move branch; DISCARD changes (dangerous)
```

When to use: - `--soft HEAD~1` — "undo my last commit but keep the changes staged" (like un-committing). - `--mixed HEAD~3` — "unwind the last 3 commits, leave the changes in my working tree, let me redo." - `--hard origin/main` — "wipe my local branch and match the remote exactly" (after a botched rebase, for example).

> **CAUTION. `GIT RESET --HARD` DISCARDS UNCOMMITTED CHANGES.** They're gone unless they were committed (in which case the reflog can recover them — see §8). For uncommitted work, stash first or use `--mixed`.

`git revert <commit>`

Creates a new commit that undoes a previous commit's changes. Doesn't rewrite history.

```
git revert abc123               # new commit that reverses abc123
git revert HEAD                 # new commit that reverses the latest commit
```

When to use: the bad commit is **already on `main` and pushed**. Reverting preserves history (audit trail) while undoing the behavior. This is the right tool for "I shipped a bug, roll it back" on a public branch.

Choosing between them

Situation	Tool
Last commit needs a small fix, not pushed	`git commit --amend`
Need to clean up the last N commits before pushing	`git rebase -i HEAD~N`
Need to "uncommit" but keep the changes	`git reset --soft HEAD~1`
Need to discard local work and match remote	`git reset --hard origin/<branch>`
Bad commit is already on `main` and pushed	`git revert <commit>`

---

6. `--force-with-lease`: the safe way to overwrite remote history

When you rewrite history that's already been pushed (typically your feature branch after a rebase), a plain `git push` will fail because the local and remote histories have diverged. You need `--force` to overwrite the remote.

Plain `--force` is dangerous. It overwrites remote history no matter what. If someone (or another machine of yours) pushed to the same branch in the meantime, that work is gone.

`--force-with-lease` is the safe version. It checks: "is the remote still at the commit I last fetched?" If yes, force-push. If no, abort — because something has changed remotely that you don't know about.

```
git push --force-with-lease
```

> **IMPORTANT. NEVER `GIT PUSH --FORCE`. ALWAYS `GIT PUSH --FORCE-WITH-LEASE`.** Even when you're sure you're solo. The lease check is essentially free and prevents the catastrophic case where you've been working on two laptops and forgot to fetch.

Even better: `--force-with-lease --force-if-includes`

Modern Git (2.30+) adds `--force-if-includes` which is even stricter: it requires that your local branch was actually based on the remote's current state (not just "I fetched at some point in the past").

```
git push --force-with-lease --force-if-includes
```

Worth aliasing in your `~/.gitconfig`:

```
[alias]
    pushf = push --force-with-lease --force-if-includes
```

Then `git pushf` is your "I rebased, now overwrite my branch on the remote, safely."

When force-push is appropriate

- **Your own feature branch**, after rebase or amend, before merging. Always OK.
- **`main`** — almost never. If you absolutely must (e.g., committed a secret), the proper procedure is: rewrite history → force-push → notify any other consumers → rotate the leaked secret because it's still in the reflog and probably in cached clones.

> **CAUTION. FORCE-PUSHING TO `MAIN` IS THE CLOSEST THING TO "PRESS TO UNDO SOLO DEV'S WHOLE DAY."** The CI/CD pipeline you built will redeploy old code on every commit, the deployed version will whiplash, and any other machine you have with a checkout of *`main`* will be confused. Avoid except in emergencies, and always *`--force-with-lease`*.

---

7. Stashing

`git stash` saves your uncommitted work somewhere safe and reverts your working tree to clean. Useful when you need to switch contexts without committing in-progress work.

```
git stash                          # save current changes, clean working tree
git stash push -m "WIP refactor"   # save with a name
git stash list                     # see saved stashes
git stash show -p                  # see what's in the most recent stash
git stash pop                      # apply the most recent stash and remove it
git stash apply stash@{2}          # apply a specific stash, keep it in the list
git stash drop stash@{0}           # delete a stash
git stash clear                    # delete all stashes
```

When to stash

- You're in the middle of a feature, urgent fix lands in your lap — stash, switch branches, fix, switch back, pop.
- You started working without making a branch, then realized you should have. Stash, `git switch -c feature/x`, pop.
- You need to pull but have local changes. (Usually `git pull --rebase` handles this; stash if it refuses.)

When NOT to stash

- **For more than a few hours.** Stashes are easy to forget about and accumulate. Commit-in-progress (with a "WIP" message, to be squashed later) is more durable.
- **Across machines.** Stashes are local-only; they don't push. Use a WIP commit if you need to take work to another machine.
- **For multiple separate contexts simultaneously.** Stashes are a stack, not a list. Multi-context juggling is what branches are for.

> **TIP.** If you find yourself stashing often, you might be missing a branching opportunity. Branches are the long-term solution; stashing is for "next 30 minutes."

---

8. `git reflog`: the safety net

The **reflog** is git's local-only record of every change to every branch tip on your machine. Every commit, rebase, reset, checkout — they all leave entries in the reflog with the previous SHA.

What this means: **almost nothing is truly lost.** Force-pushed? The previous SHA is in the reflog. Deleted a branch? Its tip SHA is in the reflog. Botched a rebase? Pre-rebase SHA is in the reflog.

```
git reflog                        # for HEAD
git reflog show feature/leaderboard   # for a specific branch
```

Output looks like:

```
abc1234 HEAD@{0}: rebase finished: returning to refs/heads/feature/leaderboard
abc1234 HEAD@{1}: rebase: Add tests
def5678 HEAD@{2}: rebase: Wire leaderboard to scores
ghi9abc HEAD@{3}: rebase: Add leaderboard skeleton
jkl0def HEAD@{4}: rebase (start): checkout main
mno1234 HEAD@{5}: commit: Add tests
pqr5678 HEAD@{6}: commit: Wire leaderboard to scores
```

Each line: SHA, ref-spec, action. `HEAD@{5}` is the state of HEAD 5 changes ago.

The two-step rescue

When something has gone wrong:

```
# 1. Find the SHA in the reflog
git reflog

# 2. Reset to it (or check it out, or cherry-pick from it)
git reset --hard <sha>
```

That's it. The "lost" state comes back because the underlying commits weren't deleted — only the branch pointer moved. Git is content-addressable storage; commits stick around until garbage collected (default: 30 days for unreachable, 90 days for reachable-but-unused).

> **NOTE. THE REFLOG IS LOCAL ONLY.** It's stored in `.git/logs/`. It doesn't survive a fresh clone, doesn't sync to remotes. If you delete your local repo, the reflog goes with it. So the safety net works on the machine where the work happened; not after `rm -rf` of the repo.

Cleaning the reflog

By default, reflog entries expire after 90 days (or 30 for entries that aren't reachable from any current ref). Running `git gc` cleans up. You can force immediate cleanup with `git reflog expire --expire=now --all && git gc --prune=now --aggressive`, but rarely a good idea — the safety net is more valuable than the disk space.

---

9. Recovery procedures

"I committed to `main` by accident"

The latest commit is on `main` but should be on a branch.

```
# 1. Create a branch at the current commit
git switch -c feature/my-work

# 2. Move main back one commit
git switch main
git reset --hard HEAD~1

# 3. Push the branch, push main as-is (it didn't change remotely if you hadn't pushed yet)
git switch feature/my-work
git push -u origin feature/my-work
```

If you already pushed the commit to `main` and need to undo it on the remote: don't. Use `git revert` instead (creates a new commit that undoes it), then carry on as if the bad commit hadn't happened.

"I deleted a branch I needed"

```
git reflog
# find a line like: abc1234 HEAD@{12}: checkout: moving from feature/lost to main
# That abc1234 is the deleted branch's tip

git switch -c feature/lost abc1234
```

"I force-pushed bad code over good code"

The good code is gone from the remote. But your local reflog probably still has the good SHA:

```
git reflog                    # find the pre-bad-push SHA
git reset --hard <good-sha>   # restore locally
git push --force-with-lease   # repush the good version
```

If you also lost the local copy (force-pushed from another machine), and this machine never had the good code, the reflog on the other machine is your only hope. Go to that machine.

"I botched a rebase, my history is a mess"

```
git reflog
# find the entry just before "rebase (start)": something like
# abc1234 HEAD@{20}: checkout: moving from main to feature/x
# That abc1234 is the pre-rebase tip of your branch

git reset --hard abc1234
```

Or, while the rebase is in progress (you're in conflict resolution and want to bail):

```
git rebase --abort
```

"I committed a secret"

If the commit hasn't been pushed:

```
# Remove the secret from the file, then:
git commit --amend          # if it was the latest commit
# or
git rebase -i <commit>      # use "edit" on the offending commit, fix, continue
```

If it has been pushed: assume the secret is compromised. **Rotate the secret immediately** — change the password, regenerate the API key, etc. Then rewrite history and force-push:

```
# Use git-filter-repo (modern, recommended) or BFG Repo-Cleaner
pip install git-filter-repo
git filter-repo --path path/to/secret-file --invert-paths
git push --force-with-lease
```

Even after force-pushing, cached clones and forks may still have the secret. Rotation is the only real fix.

"I want to redo a merge I just did"

```
git reset --hard ORIG_HEAD    # ORIG_HEAD points at where you were before the merge
```

`ORIG_HEAD` is automatically set by merge/rebase/pull operations to the pre-operation state. Useful escape hatch.

---

10. Working across multiple machines

Your laptop, your work computer, your phone. Same repo. You need to be able to switch machines without losing work.

The cardinal rule

Push before switching machines. Pull before resuming.

```
# on laptop, end of session:
git add -A
git commit -m "WIP: leaderboard, half done"
git push

# on work computer, start of session:
git pull --rebase
# continue working...
```

The WIP commit is fine; you'll squash it into the proper feature commits later. Working tree empty + pushed = you can resume anywhere.

When you forgot to push

You're at the other computer and there's uncommitted work on the laptop you left at home. Options, in order of preference:

1. **Remote in to the laptop, push from there.** Tailscale (see future how-to) or VPN.
2. **Live without that work for now.** Start new work on the other computer; merge later.
3. **Reconstruct from memory.** Last resort.

Multiple machines + force-push

This is where `--force-with-lease` really matters. If you rewrote history on machine A, force-pushed, then machine B still has the old history. On machine B:

```
git fetch origin
git reset --hard origin/feature/x      # match the remote
```

If machine B had its own unpushed work that's now incompatible, that work needs to be replayed (cherry-picked from the reflog) onto the new history. Painful but recoverable.

`git config push.autoSetupRemote`

Set this once globally:

```
git config --global push.autoSetupRemote true
```

It makes `git push` automatically set up tracking for a new branch on its first push, so you don't have to type `git push -u origin <branch>` the first time. Small QoL, compounds.

---

11. Git config worth setting once

A `~/.gitconfig` that pulls its weight:

```
[user]
    name = Joshua Julian
    email = joshuawjulian@gmail.com
    signingkey = ~/.ssh/id_ed25519.pub

[init]
    defaultBranch = main

[push]
    autoSetupRemote = true
    default = simple
    followTags = true

[pull]
    rebase = true              # always pull with rebase; no merge commits from pull

[rebase]
    autoStash = true           # auto-stash dirty WD before rebase
    autoSquash = true          # auto-detect fixup!/squash! commits

[fetch]
    prune = true               # delete remote-tracking branches when remote deletes them
    pruneTags = true

[diff]
    algorithm = histogram      # better diffs for non-trivial changes
    colorMoved = zebra         # highlight moved blocks differently from added/removed

[merge]
    conflictStyle = zdiff3     # 3-way conflict markers including the common ancestor

[rerere]
    enabled = true             # remember conflict resolutions, reapply automatically

[commit]
    gpgsign = true             # sign commits with SSH key (since Git 2.34)
    verbose = true             # show full diff in the commit message editor

[gpg]
    format = ssh               # use SSH keys for signing

[gpg "ssh"]
    allowedSignersFile = ~/.ssh/allowed_signers

[alias]
    co = checkout
    sw = switch
    rs = restore
    st = status -sb
    cm = commit -m
    am = commit --amend --no-edit
    pushf = push --force-with-lease --force-if-includes
    lg = log --graph --decorate --pretty=format:'%C(auto)%h%d %s %C(black)%C(bold)%cr %an' --
abbrev-commit
    last = log -1 HEAD --stat
    branches = branch -vv
    pristine = !git reset --hard && git clean -fdx
    undo = reset HEAD~1 --mixed
```

What each non-obvious setting does:

- **`init.defaultBranch = main`** — new repos default to `main`, not the older `master`.
- **`push.autoSetupRemote = true`** — `git push` works on a new branch without `-u origin <branch>`.
- **`pull.rebase = true`** — `git pull` rebases instead of merging. No "merge branch 'main' of github.com..." commits.
- **`rebase.autoStash = true`** — git stashes dirty changes for you before rebasing, restores after.
- **`rebase.autoSquash = true`** — commits prefixed with `fixup!` or `squash!` auto-position themselves during interactive rebase. Pair with `git commit --fixup <sha>`.
- **`fetch.prune = true`** — `git fetch` deletes the local refs for branches you've deleted from the remote. Keeps `git branch -a` clean.
- **`diff.algorithm = histogram`** — better diffs, especially for moved code.
- **`merge.conflictStyle = zdiff3`** — conflict markers show the common ancestor in addition to "yours" and "theirs," which is genuinely useful for understanding what each side changed.
- **`rerere.enabled = true`** — git remembers how you resolved a conflict and auto-applies the same resolution if it sees the same conflict again (e.g., during repeated rebases). Free magic.
- **`commit.gpgsign = true`** with `gpg.format = ssh`** — sign your commits with your SSH key. GitHub shows them as "Verified." See ssh-keys for the signing key setup.

`~/.ssh/allowed_signers`

If you're signing commits with SSH, set this up too:

```
joshuawjulian@gmail.com ssh-ed25519 AAAAC3Nz...your-public-key
```

One line per identity. Git uses this to verify signatures locally with `git log --show-signature`.

---

12. Resolving merge conflicts (the solo version)

Solo conflicts happen mostly when:

- You've been working on two computers and forgot to pull.
- You rebased a branch onto an updated `main` and your changes touched the same lines as commits on `main`.
- A long-lived branch (avoid these — §3) finally needs to merge.

The workflow

```
git rebase main         # or git merge main, but rebase is solo-default

# CONFLICT (content): Merge conflict in path/to/file.py
# Auto-merging fails. Open the file.
```

The file now has conflict markers (with `merge.conflictStyle = zdiff3`):

```
<<<<<<< HEAD
return user.scores.first().value
||||||| common ancestor
return user.scores[0]
=======
return user.scores.order_by(Score.created_at.desc()).first().value
>>>>>>> incoming change
```

Three sections:

- `<<<<<<<` to `|||||||` — the version on your current branch (HEAD).
- `|||||||` to `=======` — the common ancestor (what both versions started from).
- `=======` to `>>>>>>>` — the incoming change.

Edit the file to the version you want. Delete the conflict markers. Save.

```
git add path/to/file.py
git rebase --continue
```

Repeat for each conflicted file.

VS Code's UX

VS Code's built-in merge editor (since v1.69) handles this well: open the file, click "Resolve in Merge Editor" at the top, get a 3-pane view with click-to-accept buttons. Less error-prone than editing markers by hand.

When you're stuck

```
git rebase --abort          # bail out, go back to pre-rebase state
git merge --abort           # bail out of a merge
```

You're back where you started. Try again with a different strategy.

Common conflict types and how to resolve

Conflict type	Resolution
Both sides changed the same line	Pick one; you'll usually know which
You renamed a file; main changed the original	Apply main's change to the new path manually
Both sides added the same import	Keep one
Trailing whitespace / line-ending churn	Check `core.autocrlf` / `.gitattributes` — this should be prevented, not resolved

---

13. The `gh` CLI

`gh` is GitHub's official CLI. Install once, use everywhere:

```
# Authenticate (one time)
gh auth login

# Create a PR for the current branch
gh pr create --fill              # use the latest commit as title + body
gh pr create                      # interactive: title, body, base branch
gh pr create --web                # open in browser

# Self-review
gh pr view                        # PR metadata
gh pr diff                        # the diff in your terminal
gh pr checks                      # CI status
gh pr view --web                  # open in browser

# Merge
gh pr merge --auto --squash --delete-branch
gh pr merge 42 --rebase           # merge a specific PR with rebase

# Repo ops
gh repo clone joshuawjulian/howtos
gh repo view --web

# Issues
gh issue create
gh issue list
gh issue close 5

# Workflow runs
gh run list
gh run watch                       # tail the latest run
gh run rerun <run-id>
```

For solo dev, the big wins are:

1. `gh pr create --fill && gh pr merge --auto --squash` — the two-command "self-PR + auto-merge" combo. Auto-merge waits for CI before merging.
2. `gh run watch` — tail your latest CI run from the terminal, see when it goes green/red.
3. `gh ssh-key add ~/.ssh/id_ed25519.pub -t "my laptop"` — add SSH keys without going to the web.

> **TIP.** `gh pr create --fill --web` opens the PR page in your browser after creation. Useful if you want to add reviewers, labels, or edit the description before merging.

---

14. Alternatives considered

Workflow

- **GitFlow** — many long-lived branches, release branches, hotfix branches. Reconsider when you're working on a team with formal release cycles. Skip for solo.
- **Trunk-based development** — short branches, frequent merges, feature flags for incomplete work. This is essentially what we're doing, just without the formal name.

- **Merge commits everywhere** — every PR merges with a merge commit (no squash, no rebase). Preserves the most history. Reconsider when PR history is genuinely useful — usually team context.

Tooling

- **Magit (in Emacs)** — possibly the best Git UI ever made. Reconsider when you use Emacs.
- **GitLens (VS Code)** — annotates every line with last-modified-by and provides a graphical history. Use it. Free, low-overhead, genuinely useful inside VS Code.
- `tig` — terminal UI for git log/diff/blame. Nice for keyboard-driven exploration without leaving the terminal.
- `lazygit` — full terminal UI for git operations (stage, commit, branch, rebase). Reconsider when you'd rather press keys than type git commands. Real power tool.
- **JJ (Jujutsu)** — Git-compatible VCS with a cleaner mental model (every change is a "change," not a "commit"; no separate stash; auto-tracking of working directory). Watch this space. It's the most credible "post-Git" contender. As of 2026, still pre-1.0; promising but immature. Reconsider in a year or two.
- **Sapling** — Meta's open-source VCS with novel UX. Niche; Meta-flavored.

Hosting

- **GitHub** (this guide implicitly) — biggest network, best CI, best tooling. Default for personal.
- **GitLab** — equivalent, occasional self-hostable.
- **Codeberg / sr.ht** — privacy-focused, smaller network.
- **Self-hosted Gitea/Forgejo on your VPS** — works; the network effects of GitHub usually outweigh it for personal use. Reconsider when you care about owning your code's home.

Signing

- **SSH commit signing** (this guide) — uses your existing SSH key; simpler than GPG.
- **GPG commit signing** — older, more standard, more ceremony. Reconsider when you're in an org that requires GPG.
- **No signing** — fine for solo personal. GitHub will show your commits as "Unverified," which doesn't really matter when you're the only author. Reconsider when you ever ship anything where supply-chain attestation matters.

---

15. Quick reference

Daily workflow

```
git switch main && git pull            # start fresh
git switch -c feature/name             # branch
# ... work ...
git add -A && git commit -m "..."      # commit
git rebase -i main                     # clean up before pushing
git push -u origin feature/name        # push
gh pr create --fill                    # PR
gh pr merge --auto --squash --delete-branch   # merge
git switch main && git pull            # back to main
```

History rewriting

```
git commit --amend                     # tweak last commit
git commit --amend --no-edit           # add staged to last commit, keep message
git rebase -i HEAD~5                   # interactive rewrite of last 5
git rebase -i main                     # rewrite all commits since main
git reset --soft HEAD~1                # uncommit, keep changes staged
git reset --mixed HEAD~1               # uncommit, unstage changes
git reset --hard origin/main           # nuke local, match remote
git revert <sha>                       # new commit that undoes <sha>
git push --force-with-lease --force-if-includes   # safely overwrite remote
```

Stash

```
git stash                              # save WIP, clean working tree
git stash push -m "name"               # save with a name
git stash list                         # see stashes
git stash pop                          # apply + remove latest
git stash apply stash@{2}              # apply specific, keep
git stash drop stash@{0}               # delete specific
```

Recovery

```
git reflog                             # see recent state changes
git reset --hard <sha>                 # rewind to a specific state
git rebase --abort                     # bail mid-rebase
git merge --abort                      # bail mid-merge
git switch -c recovered <sha>          # branch from a reflog SHA
git reset --hard ORIG_HEAD             # undo the most recent merge/rebase/pull
```

Inspection

```
git status -sb                          # short status
git log --oneline -20                   # last 20 commits, one line each
git log --graph --all --oneline         # branchy graph
git diff                                # unstaged changes
git diff --staged                       # staged changes
git diff main..HEAD                     # everything since main
git show <sha>                          # full content of a commit
git blame <file>                        # who last touched each line
```

`gh` ops

```
gh pr create --fill                     # self-PR with auto-filled body
gh pr view --web                        # open PR in browser
gh pr diff                              # PR diff in terminal
gh pr checks                            # CI status
gh pr merge --auto --squash --delete-branch
gh run watch                            # tail latest CI run
gh issue create
```

Configuration (one time)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global push.autoSetupRemote true
git config --global pull.rebase true
git config --global rebase.autoStash true
git config --global rebase.autoSquash true
git config --global fetch.prune true
git config --global diff.algorithm histogram
git config --global merge.conflictStyle zdiff3
git config --global rerere.enabled true
```